main线程栈

```
UserService proxy
```

```
UserService user
UserInvocation h
```

```
String userName = proxy.findUserName( id: 12);
```

```
UserService proxy = (UserService) Proxy.newProxyInstance(
        user.getClass().getClassLoader(),
        new Class[]{UserService.class},
        h
);
```

```
public class UserInvocation implements InvocationHandler {

    public UserService userService;
```

```
public class UserServiceImpl implements UserService {
```

Main类main方法中

```
UserService user = new UserServiceImpl();
UserInvocation h = new UserInvocation(user);

UserService proxy = (UserService) Proxy.newProxyInstance(
        user.getClass().getClassLoader(),
        new Class[]{UserService.class},
        h
);

String userName = proxy.findUserName( id: 12);
log.info(userName);
}
```

```
public class UserInvocation implements InvocationHandler {

    public UserService userService;

    public UserInvocation(UserService userService) {
        this.userService = userService;
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        System.out.println("User invoking...");
        Object result = method.invoke(userService, args);
        System.out.println("User invoked...");
        return result;
    }
}
```

生成的代理对象

```
public final class $Proxy0 extends java.lang.reflect.Proxy
                           implements UserService {

    // 1. 构造方法：必须传 InvocationHandler
    public $Proxy0(java.lang.reflect.InvocationHandler h) {
        super(h);           // 调用 Proxy(InvocationHandler h)
    }

    // 2. 接口方法：全部变成 final，内部只有一句话
    public final java.lang.String findName(long id) {
        return (String) h.invoke(this,
                                 m3,            // Method 对象
                                 new Object[]{(Long.valueOf(id))})
    }

    // 3. 运行期生成的 Method 字段（只列一个）
    private static java.lang.reflect.Method m3;
    static {
        try {
            m3 = Class.forName("UserService")
                      .getMethod("findName", long.class);
        } catch (NoSuchMethodException e) {
            throw new NoSuchMethodError(e.getMessage());
        }
    }
```

```
public interface UserService {
    String findUserName(int id);
}
```

```
public class UserServiceImpl implements UserService {
    @Override  1 usage
    public String findUserName(int id) {
        return "user" + id;
    }
}
```